Mobile Applications

3. Intents and broadcast receivers in Android

Boni García

boni.garcia@uc3m.es

Telematic Engineering Department
School of Engineering

2025/2026

uc3m | Universidad Carlos III de Madrid

CC BY NC SA

Table of contents

1. Introduction
2. Intents
3. Broadcast receivers
4. Takeaways

1. Introduction

- We have learned so far:
  - Android apps are built trough building blocks called **app components**
    - Activities, services, broadcast receivers, and content providers
  - We use **Jetpack Compose** to build user interfaces (UI)
    - Declarative, state-driven toolkit based on Kotlin
  - In Jetpack Compose, activities are lightweight containers for **composables**:
    - Composables are Kotlin functions that defines pieces of the UI
    - Jetpack Compose provides a rich built-in composables collection, e.g., material components, layouts, navigation, etc.
- In this unit, we focus on:
  - **Intents** (messaging objects used to request an action)
  - **Broadcast receivers** (app component to listen to broadcast intents)

Table of contents

1. Introduction
2. Intents
   - Explicit intents
   - Intent filter
   - Implicit intents
   - Activity result
3. Broadcast receivers
4. Takeaways

2. Intents

- **Intents** are messaging objects used to request an action from one app component to other

```
val intent = Intent
```

An intent is defined as a Kotlin object

- We use intents are to **start app components** in Android (activities, services, and broadcast receivers):
  - Start an activity
    - For example, when the **app launcher** starts the main activity of other app
  - Start a service
  - Deliver a system-wide message (broadcast intents)
- We can also use intents to **send results** to the calling component

https://developer.android.com/training/basics/intents

2. Intents

- There are two types of intents in Android:

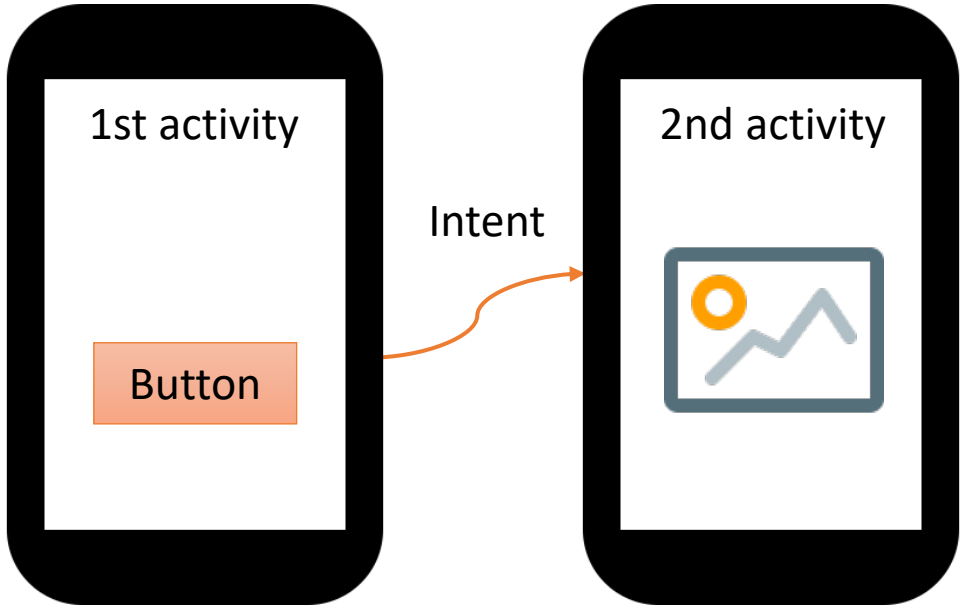
1. **Explicit intents**
   - Specify the exact target app component (by class name)
   - Used mainly within the same app
2. **Implicit intents**
   - Do not specify the target component
   - Used to request actions from other apps or system components
   - The Android system selects a matching app component (based on the **intent filters**) appropriate component to handle it

When an intent (explicit or implicit) is sent to all apps in the system, we call it **broadcast intent**

Fork me on GitHub

2. Intents - Explicit intents

- This app example uses an **explicit intent** to start a second activity:



Intent-based navigation is not usually used in Jetpack compose, since we typically use the Navigation Component for that

AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:theme="@style/Theme.MyApp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".SecondActivity"
            android:exported="false" />
    </application>

</manifest>
```

The second activity is internal to this app (`android:exported="false"`)

2. Intents - Explicit intents

Fork me on GitHub

MainActivity.kt

```
@Composable
fun MyLayout(modifier: Modifier = Modifier) {
    var text by rememberSaveable { mutableStateOf("") }
    val context = LocalContext.current

    Column(modifier = modifier) {
        Text(text = stringResource(R.string.text_msg))
        TextField(
            value = text,
            onValueChange = { text = it },
            modifier = Modifier.fillMaxWidth()
        )
        Button(
            onClick = {
                val intent = Intent(context, SecondActivity::class.java)
                intent. ("name", text)
                context.startActivity(intent)
            },
        ) {
            Text(stringResource(R.string.button_msg))
        }
    }
}
```

SecondActivity.kt

```
class SecondActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    val name = intent.getStringExtra("name")
                    val hello = String.format(stringResource(R.string.hello_msg), name)
                    MyLayout(modifier = Modifier.padding(innerPadding), hello)
                }
            }
        }
    }

}
```

The Android **context** (accesses through `LocalContext.current` in Jetpack Compose) is an object that provides access to Android functionalities and resources

The method `putExtra()` allows us to include additional data (key-value) in an intent object

2. Intents - Intent filter

- An **intent filter** is a declaration in the Android manifest (`<intent-filter>`) that specifies which **implicit intents** a component (e.g., an activity or broadcast receiver) can handle
  - For example:

```
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

- `android.intent.action.MAIN` → This activity is the entry point of the app
- `android.intent.category.LAUNCHER` → This activity is listed in the app launcher
- `android:exported="true"` → Can be launched by external components (e.g., the app launcher)

2. Intents - Implicit intents

- An **implicit intent** does not specify the exact component to start. Instead, it describes:
  - **Action**: desired operation we want to perform, represented with a string that unequivocally identified the intent
  - **Category**: additional information (e.g., the purpose or role) about the intent
  - **Data**. Optional info on which the action should be performed
- The Android system resolves implicit intents as follows:
  1. It compares the intent's action/category/data against the **intent filter** declarations in installed apps
  2. If a match is found:
     - The matching app component is started
     - The intent is delivered to it
  3. If multiple matches exist:
     - The system shows a chooser dialog

2. Intents - Implicit intents

- We use Kotlin code to specify the action/category/data as attributes of the `Intent` objects:
  - Action:

```
intent.setAction("es.uc3m.android.sendbroadcast")
intent.setAction("android.intent.action.DIAL")
```

Actions can be either custom (defined by an application) or system-defined (provided by the Android platform)

  - Category:

```
intent.addCategory("android.intent.category.DEFAULT")
```

Also, categories can be custom or system-defined

  - Data

```
intent.setData(Uri.parse("https://www.google.com"))
```

2. Intents - Implicit intents

System-defined actions are declared as string constants in the class `Intent`, e.g.:

Constant	Value	Description
ACTION_MAIN	"android.intent.action.MAIN"	Entry point for an app
ACTION_VIEW	"android.intent.action.VIEW"	Display some data to the user (e.g., using a web browser)
ACTION_DIAL	"android.intent.action.DIAL"	Dial some number to make a phone call
ACTION_ANSWER	"android.intent.action.ANSWER"	Handle an incoming phone call
ACTION_SEARCH	"android.intent.action.ACTION_SEARCH"	Perform a search
ACTION_AIRPLANE_MODE_CHANGED	"android.intent.action.AIRPLANE_MODE"	Change to airplane mode
ACTION_BATTERY_LOW	"android.intent.action.BATTERY_LOW"	Battery level <= 15%
ACTION_POWER_CONNECTED	"android.intent.action.POWER_CONNECTED"	The external power has been connected
ACTION_POWER_DISCONNECTED	"android.intent.action.ACTION_POWER_DISCONNECTED"	The external power has been removed

https://developer.android.com/reference/kotlin/android/content/Intent

2. Intents - Implicit intents

System-defined categories are declared as string constants in the class `Intent`, e.g.:

Constant	Value	Description
CATEGORY_LAUNCHER	`"android.intent.category.LAUNCHER"`	This category is used with the main activity of an application. It indicates that the activity should be listed in the **app launcher** (built-in Android app that manages the home screen and app icons)
CATEGORY_DEFAULT	`"android.intent.category.DEFAULT"`	The default category is required for implicit activity resolution
BROWSABLE	`"android.intent.category.BROWSABLE"`	This category is used with activities that can be launched from a web browser. It indicates that the activity can be safely invoked from a browser, allowing **deep linking** from web pages, i.e., custom URL schemes (e.g., `myapp://`) that, when clicked, open the app directly

https://developer.android.com/reference/kotlin/android/content/Intent

2. Intents - Implicit intents

Fork me on GitHub

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:theme="@style/Theme.MyApp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".ImplicitActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="es.uc3m.android.implicit" />
                <category android:name="android.intent.category.DEFAULT" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

The activity will be started when using the action `"es.uc3m.android.implicit"` in an implicit intent

2. Intents - Implicit intents

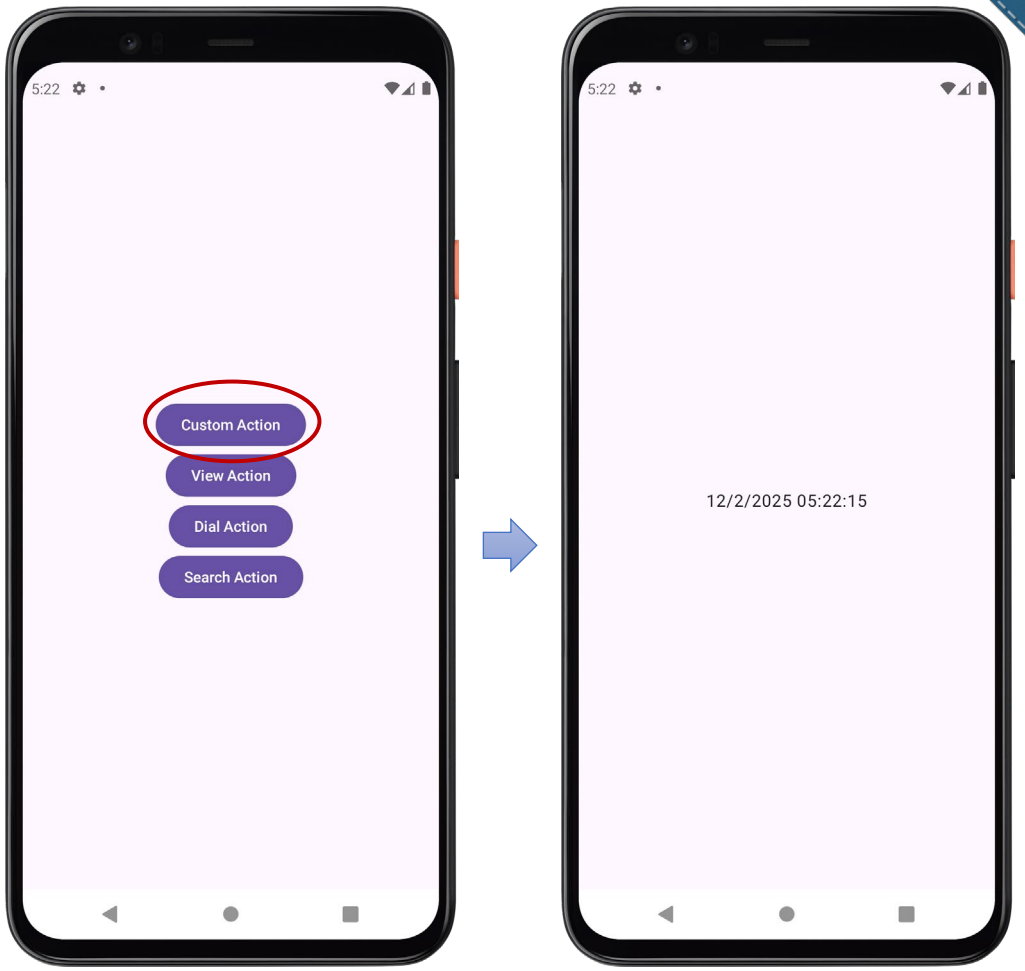
Fork me on GitHub

MainActivity.kt

```
Button(
    onClick = {
        val intent = Intent("es.uc3m.android.implicit")
        val currentDate =
            SimpleDateFormat("dd/M/yyyy hh:mm:ss", Locale.ROOT).format(Date())
        intent.putExtra("date", currentDate)
        context.startActivity(intent)
    },
) {
    Text(stringResource(R.string.button1))
}
```

ImplicitActivity.kt

```
class ImplicitActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyAppTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    val date = intent.getStringExtra("date")
                    MyLayout(String.format("%s", date))
                }
            }
        }
    }

}
```

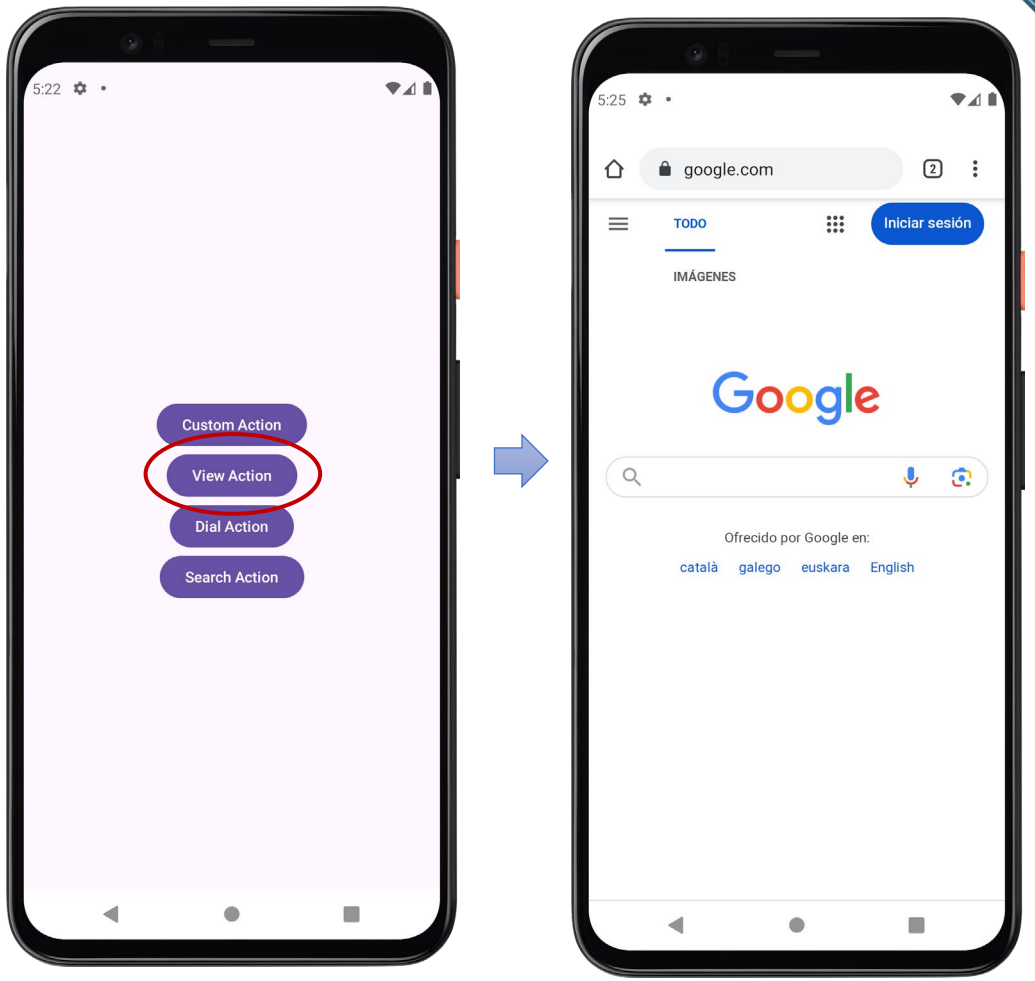


2. Intents - Implicit intents

Fork me on GitHub

```
Button(
    onClick = {
        val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com"))
        context.startActivity(intent)
    },
) {
    Text(stringResource(R.string.button2))
}
```

This app also sends several implicit intents

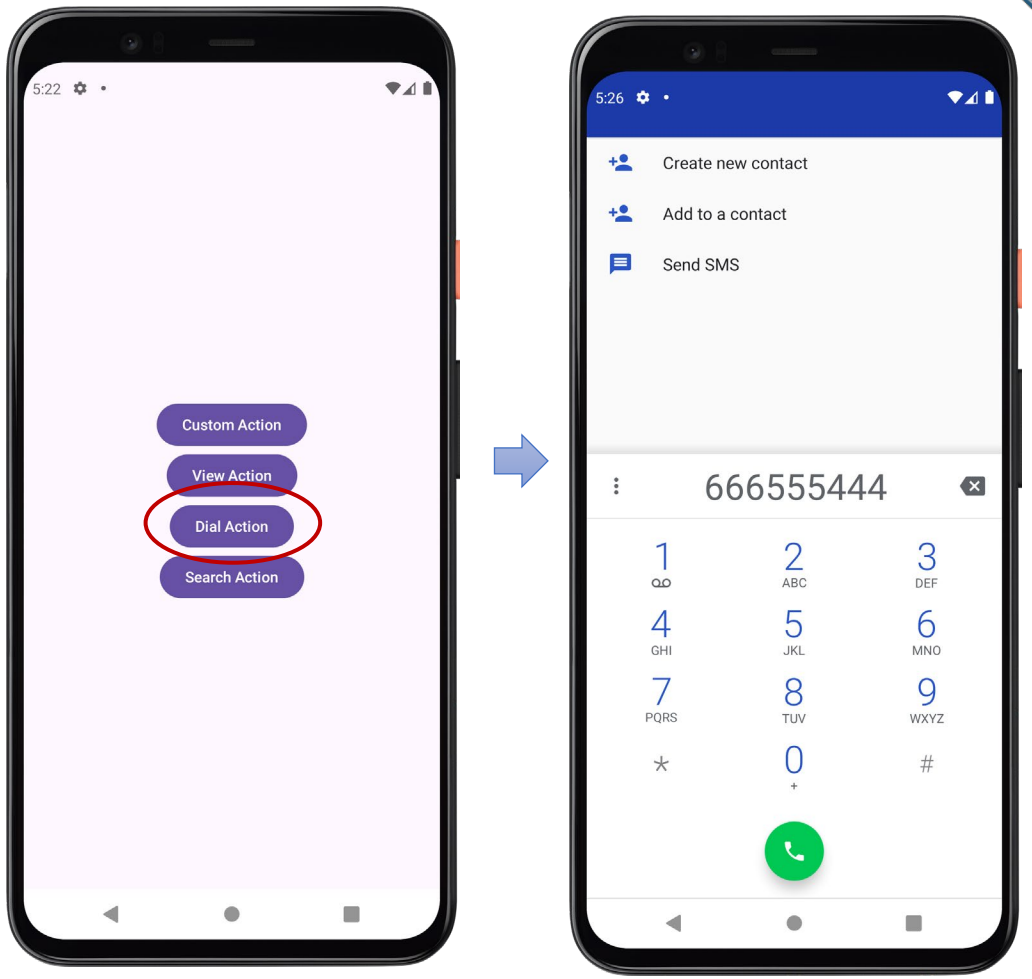


2. Intents - Implicit intents

Fork me on GitHub

```
Button(
    onClick = {
        val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:666555444"))
        context.startActivity(intent)
    },
) {
    Text(stringResource(R.string.button3))
}
```

This app also sends several implicit intents

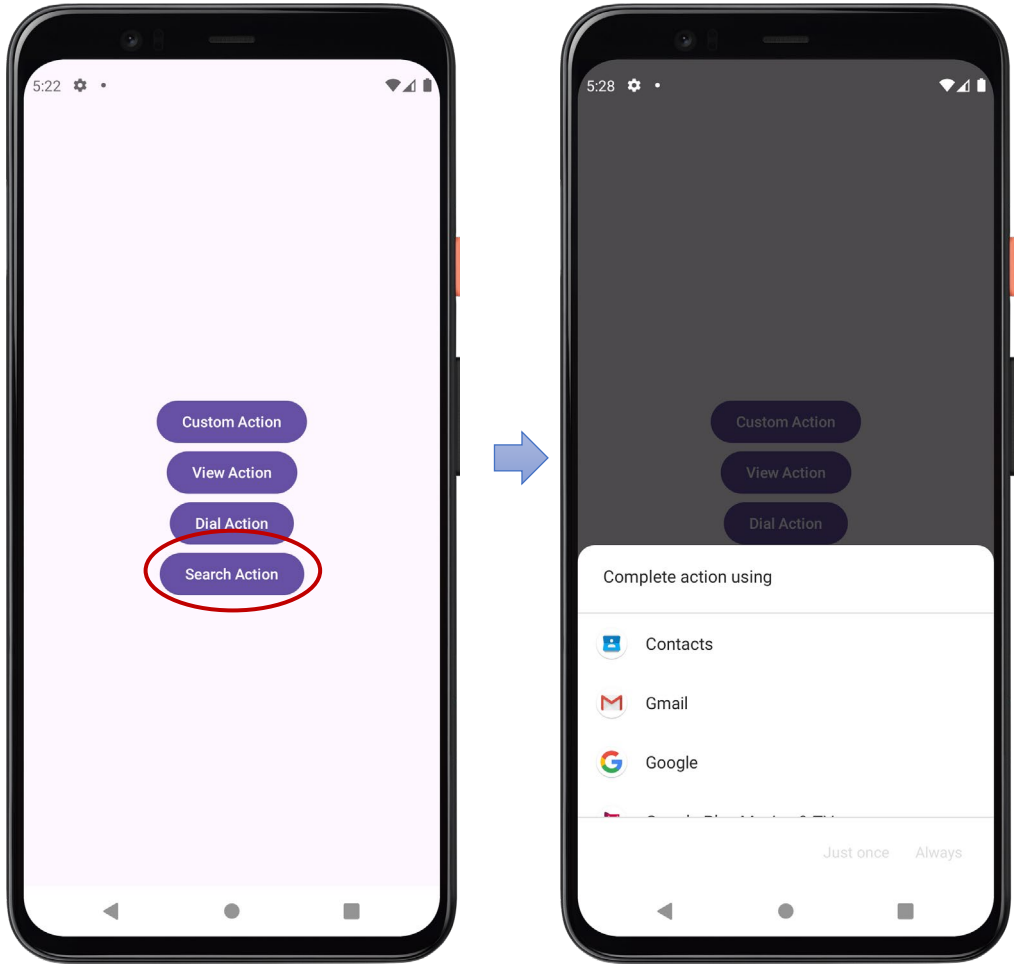


2. Intents - Implicit intents

Fork me on GitHub

```
Button(
    onClick = {
        val intent = Intent(Intent.ACTION_SEARCH)
        intent.putExtra(SearchManager.QUERY, "Developing Android apps")
        context.startActivity(intent)
    },
) {
    Text(stringResource(R.string.button4))
}
```

This app also sends several implicit intents



2. Intents - Activity result

- We can use intents to send back a result to the parent activity. This feature can be useful, for example:
  - An app can start a camera app and receive the captured photo as a result
  - An app start the Contacts app and receive the contact details as a result
- The procedure to getting results from activities is the following:
  - From the first activity:
    1. Create an instance of an activity result launcher (rememberLauncherForActivityResult)
    2. Registering a callback for receiving the result (`result ->`)
    3. Launching an intent using activity for result (`launcher.launch(intent)`)
  - From the second activity:
    4. Create a result intent (`val resultIntent = Intent()`)
    5. Define a result code (`context.setResult(resultCode, resultIntent)`)
    6. Finish the activity (`context.finish()`)

2. Intents - Activity result

Fork me on GitHub

MainActivity.kt

```
@Composable
fun MyLayout(modifier: Modifier = Modifier) {
    var text by rememberSaveable { mutableStateOf("") }
    val context = LocalContext.current
    val launcher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.StartActivityForResult()
    ) { result ->
        println("The result code is " + result.resultCode)
        val data = result.data?.getStringExtra("message")
        Toast.makeText(context, data, Toast.LENGTH_SHORT).show()
    }

    Column(modifier = modifier) {
        Text(text = stringResource(R.string.text_msg))
        TextField(
            value = text,
            onValueChange = { text = it },
            modifier = Modifier.fillMaxWidth()
        )
        Button(
            onClick = {
                val intent = Intent(context, SecondActivity::class.java).apply {
                    putExtra("name", text)
                }
                launcher.launch(intent)
            },
        ) {
            Text(stringResource(R.string.button1))
        }
    }
}
```

1. Create an instance of an activity result launcher

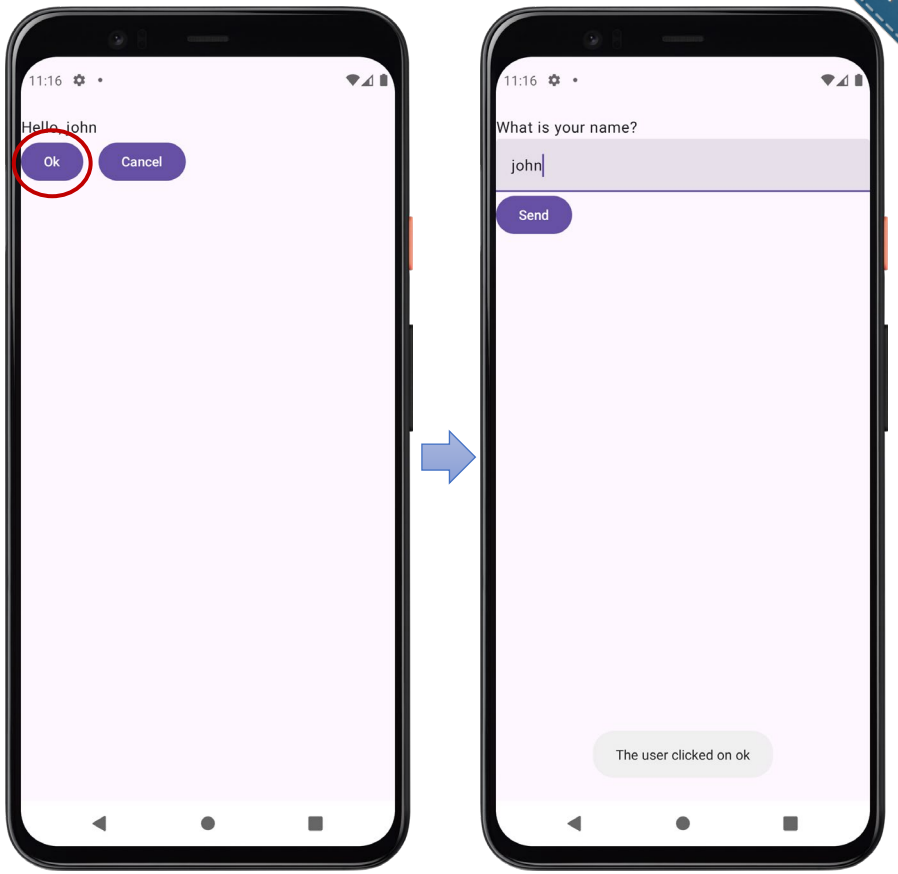
2. Registering a callback for receiving the result

3. Launching an intent using activity for result

2. Intents - Activity result

SecondActivity.kt

```
@Composable
fun MyLayout(modifier: Modifier = Modifier, text: String) {
    val okMessage = stringResource(R.string.ok_message)
    val cancelMessage = stringResource(R.string.cancel_message)
    val context = LocalContext.current

    Column(modifier = modifier) {
        Text(text = text)
        Row() {
            Button(
                onClick = {
                    if (context is Activity) {
                        finishActivity(context, okMessage, Activity.RESULT_OK)
                    }
                },
            ) {
                Text(stringResource(R.string.button2))
            }
            Button(
                modifier = Modifier.padding(start = 16.dp),
                onClick = {
                    if (context is Activity) {
                        finishActivity(context, cancelMessage, Activity.RESULT_CANCELED)
                    }
                },
            ) {
                Text(stringResource(R.string.button3))
            }
        }
    }
}

private fun finishActivity(context: Activity, resultData: String, resultCode: Int) {
    val resultIntent = Intent().apply {
        putExtra("message", resultData)
    }
    context.setResult(resultCode, resultIntent)
    context.finish()
}
```



4. Create a result intent
5. Define a result code
6. Finish the activity

Fork me on GitHub

Fork me on GitHub

2. Intents - Activity result

- A common use case for using activity results is when accesing the device camera from our app to take a picture:

```
@Composable
fun CameraCaptureScreen() {
    var imageBitmap by remember { mutableStateOf<Bitmap?>(null) }
    val context = LocalContext.current
    val cameraLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.StartActivityForResult()
    ) { result ->
        if (result.resultCode == Activity.RESULT_OK) {
            val bitmap = result.data?.extras?.get("data") as? Bitmap
            imageBitmap = bitmap
        }
    }

    // ...

}

fun launchCamera(launcher: ActivityResultLauncher<Intent>) {
    launcher.launch(Intent(MediaStore.ACTION_IMAGE_CAPTURE))
}
```

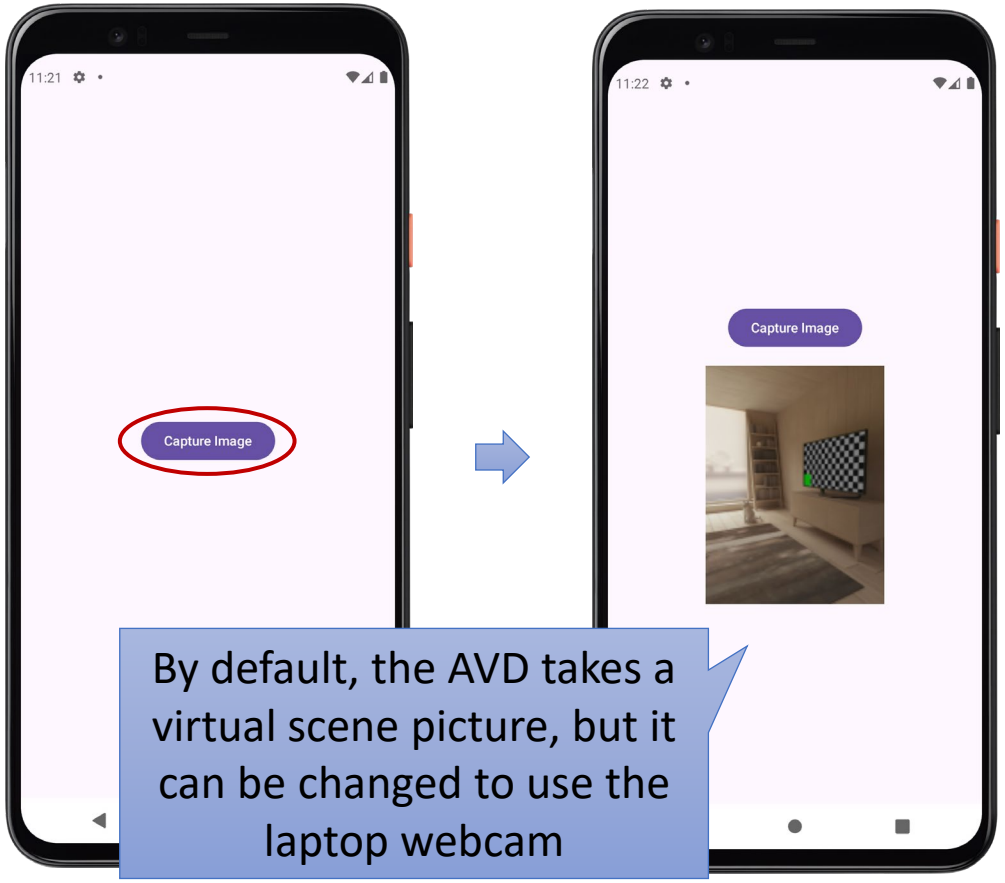


Table of contents

1. Introduction
2. Intents
3. Broadcast receivers
4. Takeaways

3. Broadcast receivers

- A **broadcast receiver** is an Android app component that:
  - Listens for broadcast intents
  - Executes code in response
  - Does not have a UI
  - Runs in the background
- A **broadcast intent** is a system-wide message sent:
  - By the system Android or another app
  - To all matching broadcast receivers

https://developer.android.com/develop/background-work/background-tasks/broadcasts

3. Broadcast receivers

- Broadcast intents are sent using the following context methods:

1. sendBroadcast()
   - Normal broadcast
   - Delivered to all matching receivers
   - No guaranteed order
2. sendOrderedBroadcast()
   - Delivered to receivers one at a time
   - Receivers are invoked in a defined order

https://developer.android.com/develop/background-work/background-tasks/broadcasts

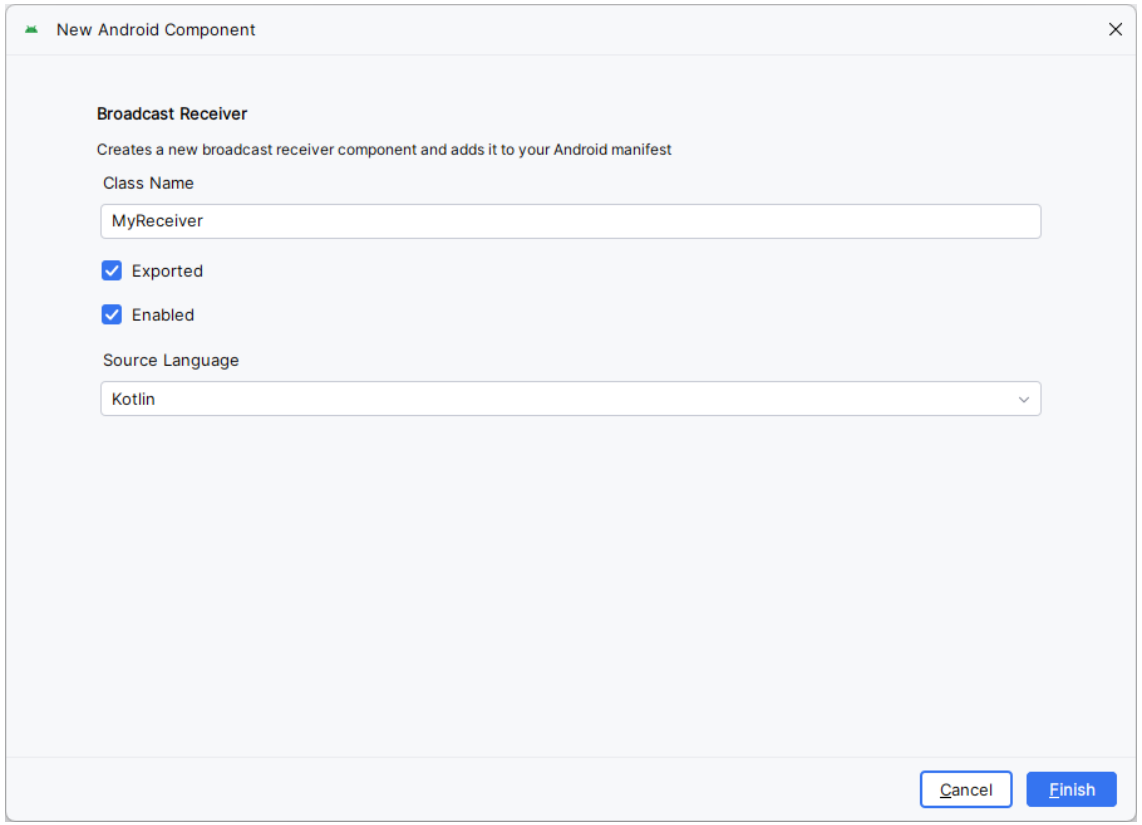
3. Broadcast receivers

- The procedure to create a broadcast receiver in an Android app is the following:

1. Create a **Java/Kotlin class** that extends `BroadcastReceiver`
2. Register this Java class in the **manifest** using an `receiver` tag
3. Declare the **intent filter** that specifies which broadcast intents the receiver should handle
   - Since Android 8 (API 26), implicit broadcasts cannot be registered in the manifest
     - They must be registered dynamically at runtime
     - This restriction improves system performance and battery efficiency

3. Broadcast receivers

- We can create an Broadcast Receiver in Android Studio using the wizard File → New → Other → Broadcast Receiver



3. Broadcast receivers

```
class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        // This method is called when the BroadcastReceiver is receiving an Intent broadcast.
        TODO("MyReceiver.onReceive() is not implemented")
    }
}
```

```
<receiver
    android:name=".MyReceiver"
    android:enabled="true"
    android:exported="true"></receiver>
```

Using that wizard, Android Studio first created the Java class for the receiver and includes a `receiver` tag in the Manifest

3. Broadcast receivers

Fork me on GitHub

```
class MainActivity : ComponentActivity() {

    private val myBroadcastIntent = "es.uc3m.android.sendbroadcast"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Register my broadcast receiver
        val receiver = MyReceiver()
        val filter = IntentFilter().apply {
            addAction(myBroadcastIntent)
            addAction(Intent.ACTION_AIRPLANE_MODE_CHANGED)
            addAction(Intent.ACTION_BATTERY_LOW)
        }

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            registerReceiver(receiver, filter, RECEIVER_EXPORTED)
        } else {
            @Suppress("UnspecifiedRegisterReceiverFlag")
            registerReceiver(receiver, filter)
        }

        enableEdgeToEdge()
        setContent {
            MyAppTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    MyScreen(action = myBroadcastIntent)
                }
            }
        }
```

The intent filter must be specified in the Java/Kotlin logic. In this example, a custom intent plus two native events will be handled by the broadcast receiver (called `MyReceiver`)

```
@Composable
fun MyScreen(action: String) {
    val context = LocalContext.current

    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Button(
            onClick = {
                val intent = Intent(action)
                context.sendBroadcast(intent)
            },
        ) {
            Text(stringResource(R.string.button))
        }
    }
}
```

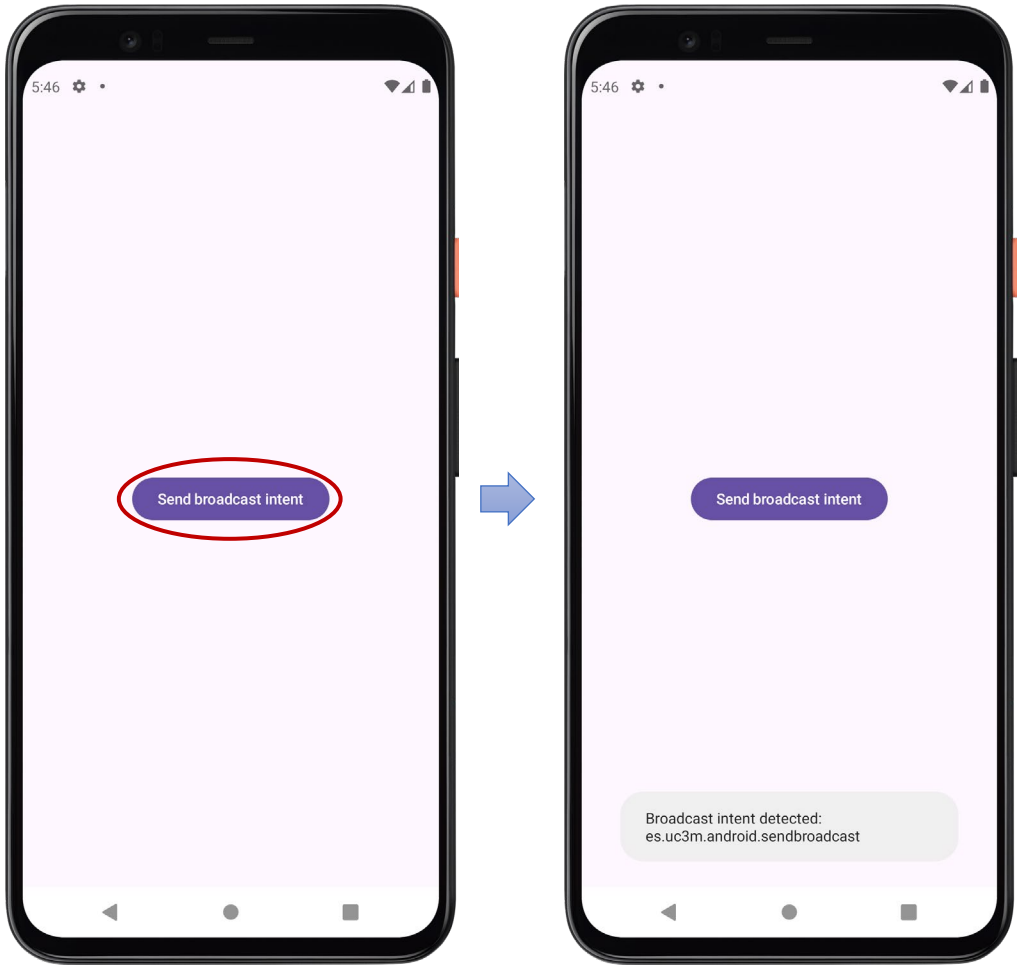
When the user clicks on the UI button, the intent with action `es.uc3m.android.sendbroadcast` is send by broadcast

3. Broadcast receivers

Fork me on GitHub

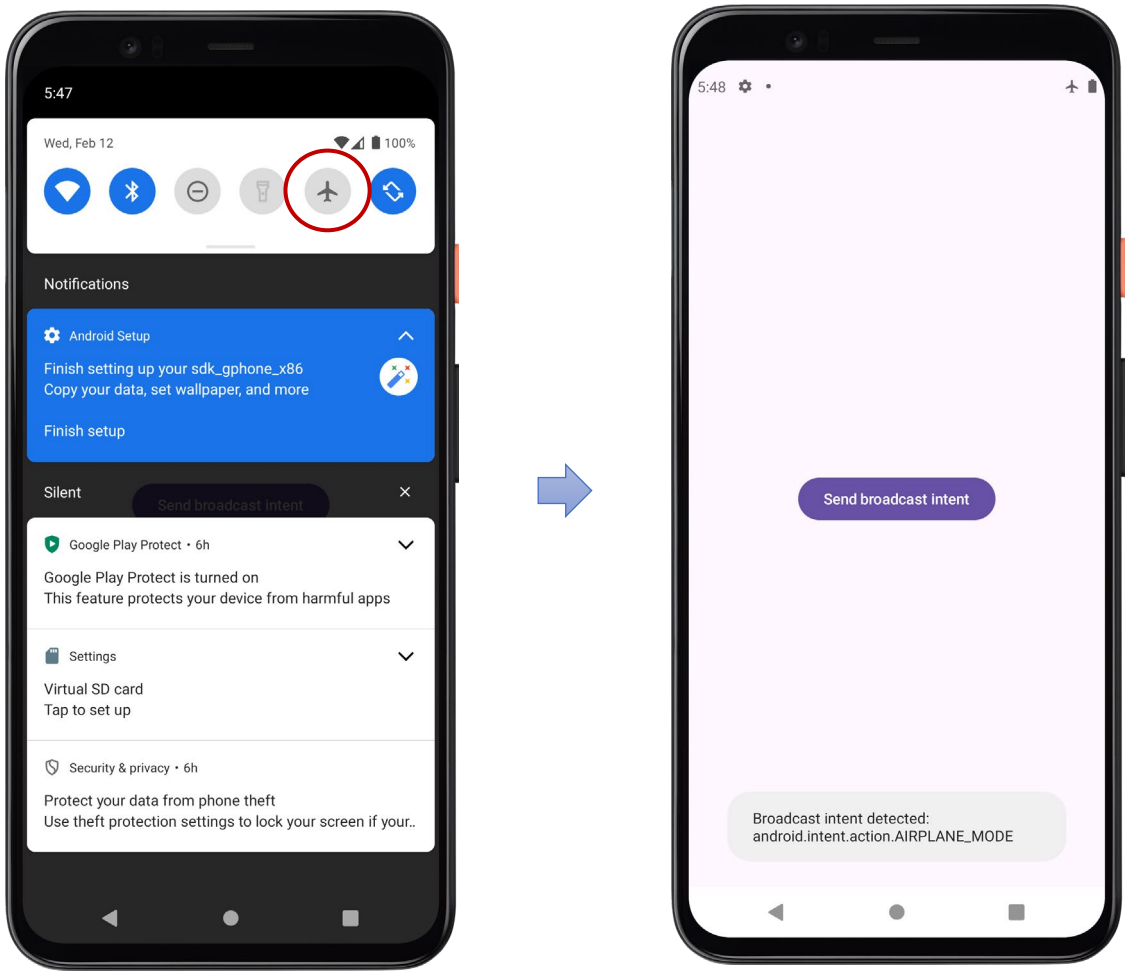
```
class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        val message = "Broadcast intent detected: " + intent.action
        Toast.makeText(context, message, Toast.LENGTH_LONG).show()
    }

}
```

The broadcast receiver simply shows a toast to the user display the intent's actions



3. Broadcast receivers

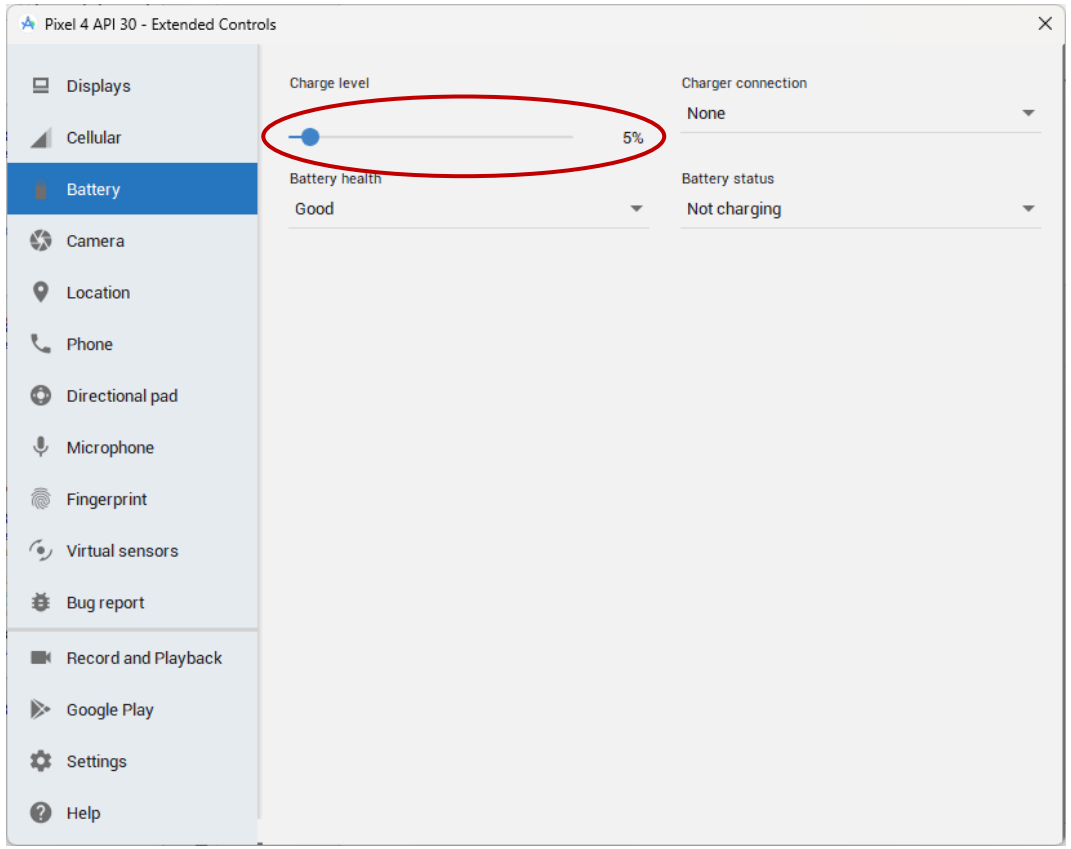
Fork me on GitHub

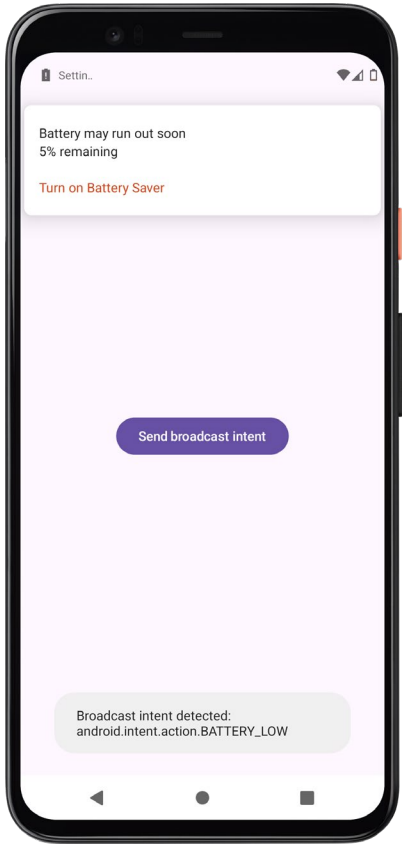


Also, when setting the airplane mode, the broadcast system intent is also received by our broadcast receiver

3. Broadcast receivers

Fork me on GitHub





We can change the battery level with using the extended controls of the virtual device

3. Broadcast receivers

- To simulate the power plug and unplug (and other options), we can use the command-line tool adb
- **Android Debug Bridge** (adb) is a tool for debugging Android devices. It is part of the Android SDK, and it is installed on:
  - Windows: `C:\Users\[user]\AppData\Local\Android\sdk\platform-tools`
  - Mac: `~/Library/Android/sdk/platform-tools/`
  - Linux: `/usr/share/android-sdk/platform-tools/`

```
adb shell dumpsys battery set level 15
```

https://developer.android.com/studio/command-line/adb

Table of contents

1. Introduction
2. Intents
3. Broadcast receivers
4. Takeaways

4. Takeaways

- Intents are messaging object we use to request an action from one app component to other
- Explicit intents are used to start activities or services defining explicitly the target component that should be invoked
- Implicit intents are used to for communication between different apps by requesting an action without specifying the target component
- A broadcast receiver is a type of app component that allows us to listen and responds to broadcast intents
- Broadcast intents are system-wide messages sent by an app or the Android system
- Android Debug Bridge (adb) is a command-line tool for debugging Android devices